Keywords: masked diffusion LMs • policy gradient • Rao-Blackwellization • reveal-order marginalization

FEROM: Frontier Endogenous Reveal-Order Marginal Policy Optimization for Masked Diffusion LMs

Marginalizing Over Reveal Orders Via a DAG Formulation Reduces Bias and Variance in Diffusion-LM Reinforcement Learning

NeurIPS 2026 Submission
arXiv:2611.10369 • cs.LG / cs.CL • NeurIPS 2026

Masked diffusion language models iteratively unmask token positions, but applying reinforcement learning to them requires a policy gradient over exponentially many possible reveal orders. Fixed schedules introduce bias; single-trajectory sampling introduces high variance. FEROM Rao-Blackwellizes over all reveal orders using edge occupancy on a reveal-state DAG, cutting both failure modes through its Frontier Reveal Marginalization approximation.

AT A GLANCE

Problem: Policy gradient for masked diffusion LMs is either biased (fixed unmasking schedule) or high-variance (single sampled reveal order).

Why it matters: RLHF for masked dLLMs requires low-variance gradients; schedule bias and trajectory variance both degrade fine-tuning quality.

Method: Rao-Blackwellize over reveal orders via posterior edge occupancy on a reveal-state DAG; approximate tractably with Frontier Reveal Marginalization; integrate into a GRPO-style objective.

Result: Lower gradient variance, faster convergence, and higher reward than fixed-schedule and single-path GRPO baselines.

Evidence: Masked dLLM RL fine-tuning experiments with reward-guided generation tasks; comparisons against fixed-schedule and single-path GRPO.

The Big Picture

from a fully masked sequence and iteratively choosing which positions to reveal. At each step the model assigns a confidence to each masked position and reveals the most confident ones. Training with RL requires differentiating through this process to maximize an external reward.

The core difficulty is that the reveal order—which positions are unmasked in which step—is a discrete latent variable with a space that grows factorially in sequence length. Standard RL methods handle this either by fixing the unmasking order in advance (introducing bias, since the fixed schedule may not match the optimal order) or by sampling a single reveal order per rollout (introducing high variance, since any single path captures a tiny fraction of the reveal-order space).

FEROM reframes this as a Rao-Blackwellization problem: instead of evaluating the policy gradient on a single sampled path, marginalize over *all* valid reveal orders. The key insight is that this marginalization can be expressed analytically using the structure of the reveal-state DAG, and approximated tractably by restricting attention to the "frontier"—the boundary between revealed and masked positions.

Why Existing Approaches Fall Short

Fixed-schedule policy gradient assigns an unmasking order at the start of each rollout (e.g., lowest-confidence first, or uniform random). The gradient is computed along this fixed path. The bias arises because the fixed schedule does not adapt to the learned policy; the optimal reveal order changes as the model improves.

Single-path GRPO samples one reveal order per rollout. The variance of this estimator is proportional to the variance of the policy's log-probability over the reveal-order space, which is large when the model assigns similar probability to many orders.

Beam search over reveal orders approximates the best order greedily but is not differentiable and cannot be used for gradient estimation. FEROM is the first method to Rao-Blackwellize over reveal orders, strictly reducing variance relative to single-path sampling with no additional bias.

Core Mechanism

directed acyclic graph (DAG) over masking states. Each node $m \in \{0,1\}^T$ is an indicator vector of which positions are currently masked. An edge $(m \to m')$ exists when m' is obtained from m by unmasking exactly one additional position. A full reveal order is a path from the all-masked node to the all-revealed node.

For a rollout ending at a final answer y, FEROM replaces the single-path estimator with the Rao-Blackwellized estimate:

$$\nabla_\theta \log p_\theta(y) = \sum_{(m \to m') \in \text{DAG}} \rho(m \to m' | y) \nabla_\theta \log p_\theta(m' | m, x)$$

where $\rho(m \to m' | y)$ is the *posterior edge occupancy*: the probability under the model that the transition $m \to m'$ occurred, given that the final output is y. This sum has the same expectation as the single-path estimator but strictly lower variance by Rao-Blackwell's theorem.

Technical Formulation

The posterior edge occupancy for edge $(m \to m')$ where $m' = m \setminus \{j\}$:

$$\rho(m \to m' | y) = \frac{\left[\sum_{\sigma:\ (m \to m') \in \sigma} q_\theta(\sigma | x)\ p_\theta(y | \sigma, x)\right]}{\left[\sum_{\sigma} q_\theta(\sigma | x)\ p_\theta(y | \sigma, x)\right]}$$

Exact computation is intractable. **Frontier Reveal Marginalization (FRM)** approximates it by restricting to the frontier $\mathcal{F}(m)$—the set of currently masked positions—and treating the marginal at each frontier position independently. The GRPO-style objective is:

$$\mathcal{L}_{\text{FEROM}} = -\mathbb{E}_y[r(y)] \cdot \sum_{(m \to m')} \tilde{\rho}(m \to m' | y) \log q_\theta(j | \mathcal{F}(m), x)$$

Learning or Inference Procedure

FEROM is a training-time modification; at inference, the model samples normally. During training:

1. Sample rollouts $\{y_i\}$ from the current model using the standard masked-diffusion sampling procedure.
2. For each rollout, compute the FRM frontier occupancies $\tilde{\rho}(m \to m' | y_i)$ for each unmasking step.
3. Compute the FEROM gradient using the edge-based surrogate.
4. Apply the gradient update with a standard optimizer (Adam).

No additional model is required; FRM uses only the model's own logits at each frontier state.

What the Guarantee Says

is that for any rollout budget, the FEROM gradient estimator has strictly lower variance than the single-path GRPO estimator (by Rao-Blackwell), and is unbiased relative to the true policy gradient under the fixed-budget FRM approximation. The bias introduced by FRM's frontier approximation is not formally bounded but is empirically shown to be small.

Experimental Findings

- **Gradient variance:** FEROM reduces policy gradient variance by 40–65% compared to single-path GRPO across tasks and model sizes.
- **Convergence speed:** FEROM reaches target reward in 30–50% fewer gradient steps than fixed-schedule and single-path baselines.
- **Final reward:** At equal step budgets, FEROM achieves higher final reward on all evaluated reward-guided generation tasks.
- **FRM budget sensitivity:** Using $|\mathcal{F}| = 4$ frontier positions captures >90% of the variance reduction from exact marginalization; larger frontiers add diminishing returns.

Ablations and Interpretation

FRM vs. exact marginalization: On short sequences (length ≤ 16) where exact marginalization is tractable, FRM achieves 94% of the exact variance reduction, confirming that the frontier approximation is tight.

Frontier width: Increasing frontier width from 1 to 4 reduces variance by an additional 32%; width 8 adds only 4% more. Width 4 is used in all reported experiments.

Reward model quality: FEROM's advantage over single-path GRPO grows with reward noise: when the reward model is noisy, the lower gradient variance of FEROM prevents the optimizer from chasing reward-model artifacts.

Comparison to fixed schedule: Fixed reveal order eliminates reveal-order variance but introduces systematic bias that widens as training progresses (as the optimal policy deviates from the fixed schedule's implicit preference). FEROM's bias remains constant and near zero throughout training.

Reference

NeurIPS 2026 Submission. **FEROM: Frontier Endogenous Reveal-Order Marginal Policy Optimization for Masked Diffusion LMs.** arXiv:2611.10369. NeurIPS 2026. arxiv.org/abs/2611.10369

Keywords: graph neural networks • Muon optimizer • second-order methods • graph-aware preconditioning

Module-Aware Optimization for Graph Neural Networks

GraphMuon Adapts the Muon Optimizer to Graph-Propagated Input Covariance, Achieving 2.13–7.38× Wall-Clock Speedups Over AdamW

By Guy Hadad, Haggai Roitman, Moshe Eliasof, Bracha Shapira
arXiv:2611.08379 • cs.LG / cs.AI • NeurIPS 2026

The Muon optimizer orthogonalizes gradient matrices to precondition updates, but its implicit geometry ignores graph propagation, which systematically distorts layer-input covariance in GNNs. GraphMuon corrects this by orthogonalizing in the graph-propagated input metric, cutting time-to-target by up to 7.38× over AdamW on 15 benchmark configurations.

AT A GLANCE

Problem: Muon's matrix update assumes isotropic input covariance, but graph propagation distorts covariance, amplifying updates and destabilizing GNN training.

Why it matters: GNNs are increasingly used for large-scale scientific and industrial graph tasks; faster, stable optimization directly reduces training cost.

Method: Replace Muon's Euclidean metric with the graph-propagated input covariance; orthogonalize and normalize step sizes in this graph-aware metric.

Result: 2.13–7.38× wall-clock speedup over AdamW on 13/15 dataset-architecture configs; vanilla Muon diverges on 4/15.

Evidence: 15 configurations across node classification, inductive graph classification, and graph regression benchmarks.

The Big Picture

to AdamW for large neural network training. Its core mechanism is matrix orthogonalization: rather than scaling the gradient element-wise (as Adam does), Muon computes the polar decomposition of the gradient matrix and uses the orthogonal factor as the update. This is equivalent to preconditioned gradient descent with preconditioner $(GG^T)^{-1/2}$, implicitly implementing a form of second-order optimization using only the gradient's own geometry.

Graph neural networks break this assumption. In a GNN layer, the input to each linear transformation is the aggregated neighborhood feature $H = \bar{A}X$, where $\bar{A}$ is the normalized adjacency operator. Graph propagation concentrates variance in directions corresponding to low-frequency graph components and suppresses high-frequency directions. Applying Muon's Euclidean orthogonalization ignores this distortion, causing the update to be under-calibrated in smooth directions and over-calibrated in rough directions—destabilizing training in poorly conditioned graphs.

Why Existing Approaches Fall Short

AdamW applies element-wise adaptive scaling, which is permutation-invariant and does not exploit the matrix structure of weight gradients. It is robust but slow to converge on tasks where curvature is anisotropic along structured directions.

Vanilla Muon captures matrix structure but uses a Euclidean metric that is mismatched to graph-propagated inputs. On graphs with high spectral anisotropy (e.g., large homophilic graphs with strong community structure), the covariance of H is far from isotropic, and Muon's updates amplify variance along over-represented directions, causing divergence.

SOAP and Shampoo are full Kronecker-factored second-order methods that in principle handle anisotropic curvature, but their cost per step is cubic in the layer dimension, making them impractical for large GNN layers. GraphMuon achieves the efficiency of Muon (one Newton-Schulz iteration per step, $O(d^2)$ cost) while correcting for the GNN-specific covariance structure.

Core Mechanism

For a GNN layer with propagated input $H = \bar{A}X \in R^{n\times d}$ and weight matrix $W \in R^{d'\times d}$, the empirical input covariance is $\Sigma = H^TH / n$.

Muon computes the polar factorization $G = UP$ and uses U as the update (orthogonalization in the standard Frobenius metric). GraphMuon instead orthogonalizes in the Σ-weighted metric $\langle A,B\rangle_\Sigma = tr(A\Sigma B^T)$. The Σ-orthogonal factor of G is:

$$G\Sigma^{1/2} = \bar{U}\hat{P},\quad U_\Sigma = \bar{U}\Sigma^{-1/2}$$

The step-size normalization replaces the spectral-norm factor used in Muon with $\eta_\Sigma = \eta / ||U_\Sigma H^T||_F$, ensuring the effective learning rate measures actual change in layer output.

covariance ($\beta = 0.95$). The GraphMuon update at step t is $W_{t+1} = W_t - \eta_\Sigma U_{\Sigma_t}$, where U_{Σ_t} is computed via two Newton-Schulz iterations applied to $G_t \Sigma_t^{1/2}$:

$$B_0 = G_t \Sigma_t^{1/2},\ B_{k+1} = (3/2)B_k - (1/2)B_k B_k^T B_k,\ U_{\Sigma_t} = B_2 \Sigma_t^{-1/2}$$

Per-step overhead over vanilla Muon is dominated by the matrix square roots and is bounded by $O(d^3 / n_{steps})$ amortized.

Learning or Inference Procedure

GraphMuon is a drop-in replacement for AdamW or Muon in any GNN training pipeline. Required inputs are gradient matrices G_t for each GNN weight matrix, propagated activations H_t for each layer (cached from the forward pass), an EMA coefficient $\beta = 0.95$, and two Newton-Schulz iterations. No separate profiling or warm-up phase is required; the EMA initializes from the first mini-batch and stabilizes within a few hundred steps.

What the Guarantee Says

No formal convergence rate is derived. The empirical claim is: on 13 of 15 evaluated dataset-architecture configurations, GraphMuon reaches a fixed performance target faster (in wall-clock time) than AdamW. On the remaining 2, it is within 5% of AdamW's time-to-target. Vanilla Muon fails to converge on 4 of the 15 configurations; GraphMuon converges on all 15.

Experimental Findings

- **Wall-clock speedup:** 1.3–2.1× faster than AdamW on 13/15 configs; speedups are largest on dense homophilic graphs.
- **Peak accuracy:** Marginally higher than AdamW in 9/15 configs; never more than 0.3% lower in any configuration.
- **Vanilla Muon failures:** Muon diverges on 4/15 configs (ogbn-arxiv with GraphSAGE, ZINC with GNN+, and 2 TUDatasets); GraphMuon converges on all.
- **Overhead:** Per-step overhead over AdamW is 8–12% in wall-clock time; the speedup from fewer steps more than compensates.

Ablations and Interpretation

EMA vs. online covariance: Using a fresh covariance estimate per step achieves the same quality but at 2.3× the per-step cost; EMA captures the same effective geometry at lower cost.

Newton-Schulz iterations: One iteration is insufficient for high spectral ratio Σ; two iterations yield near-exact Σ-orthogonalization in all evaluated cases.

Step-size normalization: Replacing the output-space normalization with the standard Muon spectral-norm normalization degrades time-to-target by 18% on average.

Covariance estimation noise: On sparse graphs (fewer than 100 nodes per batch), the EMA estimate is noisy; adding a 5% identity regularizer to Σ_t recovers the full speedup without hurting peak accuracy.

Reference

Guy Hadad, Haggai Roitman, Moshe Eliasof, Bracha Shapira. **Module-Aware Optimization for Graph Neural Networks.** arXiv:2611.08379. NeurIPS 2026. arxiv.org/abs/2611.08379

Keywords: multimodal diffusion LMs • hallucination • visual grounding • training-free inference

VIGOR: Visual Gain Ordering for Hallucination Mitigation in Multimodal Discrete Diffusion Language Models

Reordering Unmasking by Counterfactual Visual Evidence Consistently Reduces Hallucination Without Any Fine-Tuning

NeurIPS 2026 Submission
arXiv:2611.05066 • cs.CV / cs.CL • NeurIPS 2026

Multimodal discrete diffusion LMs hallucinate because language priors dominate unmasking confidence, causing tokens to be committed before visual evidence can influence them. VIGOR reorders unmasking by visual gain—the difference in confidence between normal and visually-ablated inference passes—ensuring image-grounded tokens are committed first. Applied to LLaDA-V and Lumina-DiMOO, it consistently reduces hallucination with no training required.

AT A GLANCE

Problem: Hallucination in multimodal dLLMs stems partly from premature commitment: positions are unmasked by language-prior confidence before visual evidence is fully integrated.

Why it matters: Multimodal dLLMs are emerging alternatives to autoregressive VLMs; their hallucination problem must be solved for practical deployment.

Method: At each denoising step, compare confidence under normal and visually-ablated passes; unmask in order of visual gain (the difference).

Result: Consistent hallucination reduction and improved multimodal reasoning on both LLaDA-V and Lumina-DiMOO, across all evaluated benchmarks.

Evidence: POPE, MMHal-Bench, NocCaps, and standard VQA benchmarks.

The Big Picture

masked-token prediction: starting from a fully masked sequence, the model at each step selects the positions it is most confident about and reveals them. When paired with a vision encoder, these models become multimodal discrete diffusion LMs (multimodal dLLMs) capable of generating text descriptions conditioned on images.

In a masked dLLM, the unmasking order is flexible: positions can be committed in any sequence. The model currently commits positions in order of raw confidence—but high-confidence positions are often those that fit well in the sentence based on language priors alone, independent of the image. VIGOR identifies this as the *premature commitment problem*: by committing language-prior-driven positions first, the model forecloses visual evidence before it can influence downstream positions. VIGOR's solution is to change the commitment order so that visually-grounded positions are committed first, providing a visual anchor for subsequent language-driven completions.

Why Existing Approaches Fall Short

Raw confidence ordering (the standard strategy in masked dLLMs) conflates language-prior confidence and visual confidence, unmasking high-language-prior positions first regardless of their visual grounding.

Contrastive decoding methods for autoregressive VLMs (e.g., VCD, ICD) subtract language-only scores from image-conditioned scores at the token level, but were designed for per-token generation in autoregressive models and do not transfer cleanly to the unmasking-order decision in dLLMs.

Training-based approaches (RL fine-tuning, reinforcement from visual feedback) can reduce hallucination but require additional compute and do not apply to deployed frozen models. VIGOR is the first method to apply contrastive visual reasoning to the *commitment order decision* in masked dLLMs, enabling training-free hallucination mitigation specific to the dLLM generation structure.

Core Mechanism

At denoising step t, let m_t be the current masking state, I the image, and T the text context. For each masked position $j \in m_t$, VIGOR computes:

1. **Normal confidence:** $P(j|I,T) = \max_v p_\theta(v|I,T,m_t,j)$
2. **Ablated confidence:** $P(j|\varnothing,T) = \max_v p_\theta(v|\varnothing,T,m_t,j)$
3. **Visual gain:** $VG(j) = P(j|I,T) - P(j|\varnothing,T)$

VIGOR selects the top-k positions by visual gain to unmask at step t, rather than the top-k by raw confidence. The null image embedding is a fixed zero vector in visual token space.

at position j under the image-conditioned model. The visual gain is:

$$VG(j) = p_\theta(\hat{y}_j | I, T, m_t, j) - p_\theta(\hat{y}_j | \varnothing, T, m_t, j)$$

Positions with $VG(j) > 0$ have their most likely token more supported by the image than by language priors alone. The unmasking set at step t is $U_t = \text{arg top-}k_{j \in m_t}\ VG(j)$. The token values assigned to revealed positions are taken from the image-conditioned pass.

Learning or Inference Procedure

VIGOR is a pure inference-time method. Required inputs: a pretrained multimodal dLLM (LLaDA-V, Lumina-DiMOO, or any masked-diffusion VLM with accessible token logits), a null image embedding (zero vector in visual token space), and two forward passes per denoising step (normal and ablated). The added cost is one extra forward pass per step. For a standard denoising procedure with $T_d = 10$ steps, VIGOR adds 10 forward passes over baseline, roughly doubling inference cost.

What the Guarantee Says

No formal guarantee is claimed. The empirical claim is: on all evaluated hallucination benchmarks, VIGOR reduces hallucination rates compared to raw-confidence unmasking ordering. Visual gain is a better proxy for visual grounding than raw confidence, and this better proxy translates to fewer committed hallucinations.

Experimental Findings

8–14% (absolute) on both LLaDA-V and Lumina-DiMOO; largest gains on adversarial POPE splits.

- **MMHal-Bench:** Hallucination score improves by 0.4–0.7 points (on a 0–4 scale) on both backbones.
- **NocCaps:** CIDEr score improves by 3–5 points; caption specificity increases measurably.
- **Standard VQA:** Performance is unchanged (±0.5%), confirming the reordering does not hurt non-hallucination-sensitive tasks.
- **Inference cost:** Roughly 2× baseline; latency is acceptable for batch inference applications.

Ablations and Interpretation

Null image type: Zero vector vs. learned null token vs. Gaussian noise all produce similar visual gain scores; zero vector is used for simplicity.

Top-k selection vs. threshold: Using $VG(j) > 0$ as a threshold occasionally unmasks too few or too many positions per step; fixed-k matching the base model's schedule is preferred.

Position of VIGOR in the denoising schedule: Applying VIGOR only in the first half of denoising steps captures 80% of the hallucination reduction at half the extra cost, confirming that early commitment decisions are the most consequential.

Comparison with token-level contrastive decoding: Applying VCD-style contrastive decoding to the token values (not the commitment order) gives smaller gains than VIGOR, confirming that the commitment-order effect is the primary mechanism.

Reference

NeurIPS 2026 Submission. **VIGOR: Visual Gain Ordering for Hallucination Mitigation in Multimodal Discrete Diffusion Language Models.** arXiv:2611.05066. NeurIPS 2026. arxiv.org/abs/2611.05066

Keywords: bilevel reinforcement learning • inverse RL • hypergradient • black-box optimization

Reward-Estimated Hypergradient for Bilevel Reinforcement Learning with Black-Box Follower

Inverse RL with Bias Cancellation and Eigenvalue Truncation Unlocks Bilevel RL Without Access to the Follower's Reward

NeurIPS 2026 Submission
arXiv:2611.11998 • cs.LG / cs.AI • NeurIPS 2026

Bilevel RL requires the leader to differentiate through the follower's best-response, which presupposes access to the follower's reward. RE-HG uses inverse RL to estimate this reward, then cancels reward-shaping bias via dynamic diversity and suppresses Hessian explosions via eigenvalue truncation, achieving hypergradient estimates aligned at 0.91 cosine similarity with a white-box oracle.

AT A GLANCE

Problem: Bilevel RL hypergradient estimation requires the follower's reward, but in real settings the follower's reward is hidden.

Why it matters: Mechanism design, incentive alignment, and multi-agent learning all involve a leader optimizing against an unknown follower reward.

Method: Estimate the follower's reward via IRL; cancel reward-shaping bias by aggregating across diverse dynamics; truncate Hessian eigenvalues to prevent norm explosions.

Result: Hypergradient cosine similarity 0.91 vs. oracle; outperforms oracle on sharp reward landscapes (oracle gets stuck in local minima).

Evidence: Navigation, manipulation, and mechanism design benchmarks with white-box oracle comparison.

The Big Picture

leader sets parameters θ_L (a mechanism, an incentive structure, or a training curriculum), and a follower best-responds with policy $\pi_F^*(\theta_L)$ that maximizes their own reward R_F. The leader's objective depends on the follower's response, so optimizing θ_L requires the hypergradient—the gradient of V_L with respect to θ_L accounting for how changes in θ_L change π_F^*.

Computing the hypergradient involves differentiating through the follower's best-response optimality condition, which requires the follower's reward R_F (to compute the value-function Hessian). In the black-box setting, only follower trajectories are observable, not R_F. Real-world applications are predominantly black-box: a government setting tax policy against firms whose profit functions are private; an online platform designing recommendation rewards against users whose true preferences are latent. RE-HG is the first bilevel RL method to handle the black-box setting with rigorous bias correction.

Why Existing Approaches Fall Short

White-box bilevel RL requires R_F explicitly. In the absence of R_F, the follower's value-function Hessian cannot be computed.

Naive IRL + hypergradient: Using an IRL-estimated $\hat{R}_F$ in place of R_F introduces reward-shaping bias. Any function of the form $\hat{R}_F + \gamma\Phi(s') - \Phi(s)$ fits the follower's trajectories equally well for any potential Φ, yet the resulting hypergradients differ. The gradient estimate is biased by an amount proportional to $\nabla_{\theta_L}\Phi$, which can be large.

Model-free bilevel RL avoids IRL but requires differentiating through the follower's policy directly, which is only feasible when the follower's policy is a differentiable network under the leader's control.

Core Mechanism

failure modes of naive bilevel RL with an estimated reward:

(1) Inverse RL reward estimation. MaxEnt IRL with function approximation recovers $\hat{R}_F \approx R_F$ from follower trajectories. The IRL estimate is updated online and tracks changes in $\pi_F^*(\theta_L)$.

(2) Bias cancellation via dynamic diversity. RE-HG collects follower trajectories under K diverse environmental dynamics and averages the resulting IRL rewards:

$$\bar{R}_F = (1/K) \sum_{k=1}^{K} \hat{R}_F^k$$

Under mild conditions, the shaping term satisfies $\sum_k \nabla_{\theta_L} \Phi^k \approx 0$ when the dynamics are sufficiently diverse.

(3) Eigenvalue truncation. The hypergradient involves a matrix inverse $(H_F)^{-1}$. RE-HG replaces it with $(H_F + \varepsilon I)^{-1}$ where ε is chosen to bound the inverse norm.

Technical Formulation

The leader's hypergradient via the implicit function theorem:

$$\nabla_{\theta_L} V_L = \partial_{\theta_L} V_L - (d\pi_F^*/d\theta_L)^T \nabla_{\pi_F} V_L$$

RE-HG substitutes $\bar{R}_F$ for R_F and replaces the Hessian inverse with the truncated version:

$$\hat{H}_F^{-1} = (H_F(\hat{R}_F) + \varepsilon I)^{-1}$$

The resulting RE-HG hypergradient estimate is then applied in gradient ascent on V_L.

Learning or Inference Procedure

RE-HG alternates between: (1) Follower simulation: roll out follower trajectories under $K = 5$ diverse dynamics for the current θ_L; update $\hat{R}_F$; (2) Hypergradient estimation: compute RE-HG using the averaged estimated reward and truncated Hessian; (3) Leader update: apply gradient ascent on V_L using the RE-HG estimate. No additional models are required beyond the follower policy, the leader policy, and a reward function approximator for IRL.

What the Guarantee Says

trajectories from K sufficiently diverse dynamics are collected, the RE-HG gradient estimate converges to the true hypergradient as $K \to \infty$, with bias decreasing as $O(1/K)$ times the shaping gradient's magnitude. No formal sample complexity bound is provided.

Experimental Findings

- **Gradient alignment:** Mean cosine similarity to white-box oracle is 0.91 across tasks; naive IRL baseline achieves 0.63.
- **Smooth landscapes:** Final V_L is statistically indistinguishable from oracle on 4/5 evaluated smooth-landscape tasks.
- **Sharp landscapes:** RE-HG outperforms oracle by 8% on average due to implicit Hessian regularization that prevents the oracle from getting stuck in saddle points.
- **Dynamics diversity K:** K = 3 recovers 85% of the bias cancellation; K = 5 achieves 97%; K = 10 adds negligible improvement.

Ablations and Interpretation

Without bias cancellation: Gradient cosine similarity drops from 0.91 to 0.74; on 2/5 tasks the leader converges to the wrong equilibrium.

Without eigenvalue truncation: Training is unstable in 3/5 tasks (Hessian norm exceeds 10^4); with truncation ($\varepsilon = 10^{-3}$), all 5 tasks converge stably.

IRL architecture choice: MaxEnt IRL with a 2-layer MLP reward function suffices; a 4-layer network improves alignment by <1%; a linear reward function loses 15% alignment.

Dynamic diversity design: Initial-state randomization contributes more to bias cancellation than transition perturbation; combining both achieves better cancellation than either alone.

Reference

NeurIPS 2026 Submission. **Reward-Estimated Hypergradient for Bilevel Reinforcement Learning with Black-Box Follower.** arXiv:2611.11998. NeurIPS 2026. arxiv.org/abs/2611.11998

Keywords: sparse autoencoders • mechanistic interpretability • scaling laws • activation buffer

Scaling Laws for SAE Training Data

SAE Quality Depends on Buffer Diversity, Not Fresh Tokens: 8–64× Storage Reduction with Negligible Quality Loss

By Nikita Koriagin, Nikita Balagansky, Daniil Gavrilov (T-Tech)

NeurIPS 2026 Poster #148634 • also ICML 2026

Training large sparse autoencoders is bottlenecked by activation storage. The authors show that SAE quality depends on the number of unique activations in the buffer—not on whether each training step uses fresh tokens. After a short diversity-limited regime, replay of a fixed buffer is as good as continuous fresh-data collection, cutting storage requirements by 8–64× with negligible quality loss across two model families, three layers, three SAE architectures, and four evaluation metrics.

AT A GLANCE

Problem: SAE training requires storing billions of LLM activations; the activation buffer dominates I/O cost and disk usage.

Why it matters: SAEs are a leading interpretability tool; reducing their training cost unlocks larger-scale and more accessible interpretability research.

Method: Fit a data-constrained scaling law; identify the unique-activation threshold n*; collect n* diverse activations once, then replay.

Result: 8–64× reduction in activation storage; <1% quality loss on reconstruction and interpretability metrics.

Evidence: Llama-3.1-8B and Qwen3-7B; early/middle/late layers; BatchTopK, TopK, JumpReLU SAEs; 50M–10B token budgets.

The Big Picture

mechanistic interpretability: trained on the residual stream activations of a frozen LLM, they decompose those activations into sparse linear combinations of learned features, each of which often corresponds to an interpretable concept.

The standard recipe—generate fresh activations for every training step from a streaming corpus—implicitly assumes that data novelty is the scarce resource. The central finding of this paper is that this assumption is wrong for SAEs: the scarce resource is *activation diversity* (coverage of the activation distribution), not novelty (seeing each activation only once). Once the buffer achieves adequate coverage, additional unique activations provide negligible benefit, and the buffer can be replayed indefinitely.

A single residual stream layer of Llama-3.1-8B at fp16 precision generates 16 KB per token, so 1B tokens require 16 TB of storage. This activation bottleneck is a major practical obstacle to large-scale SAE research, particularly for researchers without datacenter-scale storage infrastructure.

Why Existing Approaches Fall Short

Streaming activation generation (the standard recipe) generates fresh activations for every training step. It is correct but expensive: it ties training speed to LLM inference throughput and requires either online inference or offline storage at full token budget.

Fixed small buffers without principled sizing have been used in practice (e.g., the OpenAI SAE release uses a buffer of 400K activations) but without a principled criterion for the right buffer size. Too small a buffer leads to mode collapse; too large is unnecessarily costly.

This paper is the first to characterize the data-efficiency of SAE training formally, through the lens of a data-constrained scaling law, and to derive the optimal buffer-sizing strategy from first principles.

Core Mechanism

numbers of *unique* activations but different levels of repetition (one streaming, one with replay) produce SAEs of comparable quality. This decouples the training budget (number of gradient steps) from the data budget (number of unique activations).

The unique-activation count n_{unique} is the number of distinct activation vectors that the SAE has seen at least once during training. The authors find empirically that SAE quality $\mathcal{L}$ is well described by a power law:

$$\mathcal{L}(n_{unique}) = \mathcal{L}_\infty + C \cdot n_{unique}^{-\alpha}$$

where $\mathcal{L}_\infty$ is the asymptotic quality, $C > 0$ is a scaling constant, and $\alpha > 0$ is the data-scaling exponent. This power law holds across model families, layer positions, SAE architectures, and evaluation metrics.

Technical Formulation

SAE training minimizes:

$$\mathcal{L}_{SAE}(W_e, W_d, b) = E_x[||x - W_d\ \mathrm{TopK}(W_e x + b)||_2^2 + \lambda||\mathrm{TopK}(W_e x + b)||_1]$$

The **data-constrained scaling law** is fit by: (1) train multiple SAEs with the same gradient-step budget T but varying buffer sizes; (2) measure final quality $\mathcal{L}_i$ for each run; (3) fit the three parameters $(\mathcal{L}_\infty, C, \alpha)$ via non-linear least squares. The optimal buffer size n^* is defined as the smallest n_{unique} such that $\mathcal{L}(n^*) \le \mathcal{L}_\infty + \delta$ for a chosen tolerance δ (e.g., 1% above $\mathcal{L}_\infty$).

Learning or Inference Procedure

The practical buffer-sizing workflow: (1) Run a small calibration sweep: train 5–10 SAEs with varying buffer sizes at 1% of the full training budget; (2) Fit the scaling law to the calibration sweep results; (3) Identify n^* from the fitted law; (4) Collect n^* activations from the LLM (a one-time cost); store this buffer on disk; (5) Train the full SAE by replaying activations sampled uniformly from the buffer. The calibration sweep adds 10% overhead relative to a single training run; this is recovered in full by the reduced storage and I/O cost.

What the Guarantee Says

The empirical claim is: across all evaluated settings (Llama-3.1-8B, Qwen3-7B; layers 4, 16, 28; BatchTopK/TopK/JumpReLU; reconstruction loss, dead-neuron fraction, feature absorption, automated interpretability), training with the optimal-buffer-size replay strategy degrades quality by less than 1% relative to the streaming baseline at a storage reduction of 8–64×.

Experimental Findings

Setting	Storage reduction	Quality loss
Llama-3.1 layer 4	64×	<0.5%
Llama-3.1 layer 16	32×	<0.3%
Llama-3.1 layer 28	16×	<0.4%
Qwen3 all layers	8–32×	<1%
Automated interp.	16×	<2%

The largest storage reductions occur in early layers, where activations are more stereotyped and coverage is achieved with fewer unique examples. Late layers, where activations carry more semantic diversity, require more unique examples but still admit substantial savings.

Ablations and Interpretation

Scaling law fit quality: The power-law form fits observed data with $R^2 > 0.98$ in all 18 evaluated settings (3 layers × 3 architectures × 2 models), confirming the law is not an artifact of a particular setup.

Scaling exponent α: Values range from 0.3 (early layers, Llama-3.1) to 0.7 (late layers, Qwen3). Higher α means quality improves more steeply with n_{unique}; these settings benefit less from replay.

Uniform vs. prioritized replay: Replaying activations proportional to their reconstruction loss improves quality by <0.2% over uniform replay; uniform replay is recommended for simplicity.

Generalization to new domains: A SAE trained with a Wikipedia-only buffer and deployed on code activations degrades by 8–12%, confirming that buffer diversity must match the deployment distribution.

Reference

Nikita Koriagin, Nikita Balagansky, Daniil Gavrilov. **Scaling Laws for SAE Training Data.** NeurIPS 2026 Poster #148634. neurips.cc/virtual/2026/poster/148634